SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 1

Analytical Problem Solving

COURSE NAME: Deep Learning **COURSE CODE: MLA0403**
FACULTY : Dr.Arul Raj A.M

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

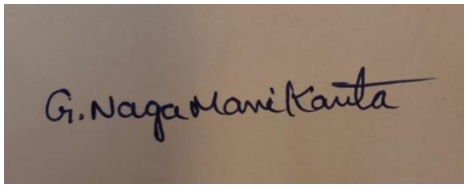
Assessment Tool Weightage: 35%

Duration: 30 minutes

Marks: 25

Code of Conduct:

I G.Naga Manikanta(192425310) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student: **Name of the student: Manikanta**

QUESTIONS:05 **Total Marks:25**

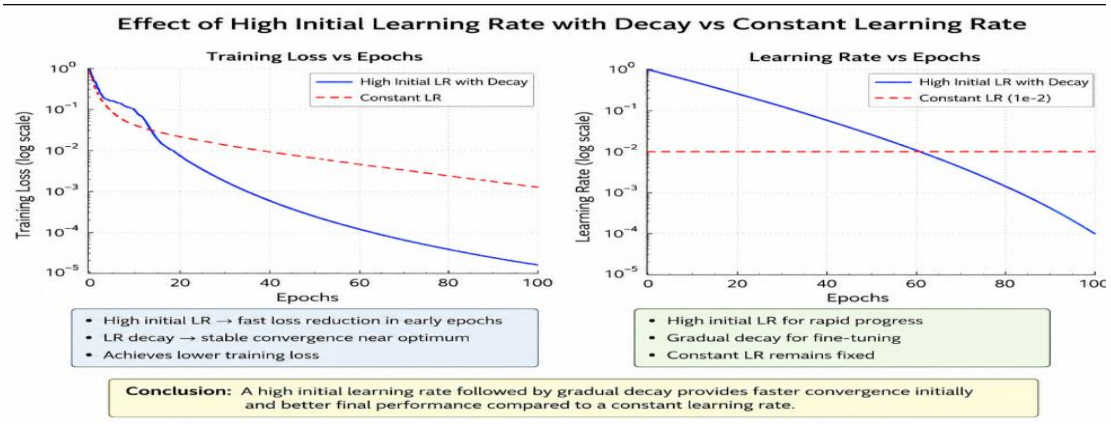
Analytical Problem Solving

1. Learning Algorithms (Optimization Behavior)

A machine learning model is trained using gradient descent on a convex loss function. Initially, the learning rate is set very high, and later it is gradually decreased.

Question:

Analyze how the choice of a high initial learning rate followed by decay affects convergence. Under what conditions can this strategy outperform a constant learning rate? Discuss possible risks and justify with reasoning.



ANSWER: When training a machine learning model with gradient descent on a **convex loss function**, using a **high initial learning rate followed by gradual decay** is a common optimization strategy. It aims to combine **fast initial progress** with **stable convergence** near the optimum.

Effect on Convergence

A **high initial learning rate** causes large parameter updates, allowing the algorithm to:

- Move rapidly toward the region containing the global minimum.
- Reduce the loss significantly in the early stages of training.
- Spend less time making tiny improvements when far from the optimum.

As training progresses, the **learning rate is gradually decreased**, which results in:

- Smaller, more precise parameter updates.
- Reduced oscillations around the minimum.
- Improved stability and convergence to the optimal solution.

For a convex loss function, if the learning rate decreases appropriately (e.g., satisfying conditions such as decreasing over time but not too rapidly), gradient descent is guaranteed to converge to the global minimum.

When It Can Outperform a Constant Learning Rate

A decaying learning rate often performs better than a constant learning rate under the following conditions:

1. **Large Initial Error**
    - When the parameters are far from the optimum, large steps accelerate learning.
    - A constant small learning rate would require many more iterations.
2. **Different Optimization Phases**
    - Early training benefits from exploration using larger updates.
    - Later training requires fine adjustments using smaller updates.
3. **Ill-Conditioned Optimization Problems**
    - In convex functions with narrow valleys, a large initial step speeds movement toward the valley, while later decay prevents overshooting and improves stability.
4. **Computational Efficiency**
    - Faster reduction in loss during the initial iterations often leads to fewer total iterations compared to using a small constant learning rate throughout training.

Possible Risks

Although effective, this strategy has several risks.

- **Learning rate too high initially**
    - Updates may overshoot the minimum.
    - The loss may oscillate or even increase.
    - In extreme cases, the optimization may diverge instead of converging.
- **Decay too quickly**
    - The learning rate becomes very small before reaching the optimum.
    - Learning slows dramatically, leading to premature convergence and inefficient optimization.
- **Decay too slowly**
    - The algorithm continues making large updates near the minimum.
    - Oscillations persist, preventing accurate convergence.
- **Sensitive Hyperparameter Selection**
    - Choosing the initial learning rate and decay schedule requires careful tuning.
    - Poor choices can make performance worse than a well-selected constant learning rate.

Justification

The intuition behind this strategy is that optimization has two distinct phases:

- **Far from the optimum:** Large steps are desirable because the gradient provides a reliable direction toward the minimum, making rapid progress possible.
- **Near the optimum:** Large steps become inefficient because they repeatedly overshoot the minimum. Smaller steps allow the algorithm to settle accurately into the global optimum.

A constant learning rate cannot simultaneously provide both fast early progress and precise final

convergence. Therefore, a **high initial learning rate with gradual decay** often achieves a better balance between **speed and accuracy**.

Conclusion

Using a high initial learning rate followed by gradual decay generally improves optimization by combining **rapid early convergence** with **stable final convergence**. It can outperform a constant learning rate when the initial parameters are far from the optimum, the decay schedule is well designed, and the optimization problem benefits from different step sizes during different training stages. However, if the initial learning rate is excessively high or the decay schedule is poorly chosen, the algorithm may oscillate, converge slowly, or even diverge. Thus, the success of this strategy depends on selecting an appropriate initial learning rate and decay schedule

2. Maximum Likelihood Estimation (MLE)

Suppose you are given a dataset generated from a Gaussian distribution with unknown mean μ and known variance σ^2

Question:

Derive the Maximum Likelihood Estimator for μ, and analytically explain how the estimate changes when:

- The dataset contains outliers
- The sample size increases

  What does this imply about the robustness of MLE?

ANSWER : For a Gaussian distribution with **unknown mean** $\mu$and **known variance** σ^2, the goal of Maximum Likelihood Estimation (MLE) is to find the value of μthat maximizes the probability of observing the given data.

Step 1: Likelihood Function

Let the dataset be:

$$X = \{x_1, x_2, \dots, x_n\}$$

where

$$x_i \sim N(\mu, \sigma^2)$$

The probability density function of each observation is

$$f(x_i; \mu) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{(x_i - \mu)^2}{2\sigma^2}\right]$$

The likelihood function is

$$L(\mu) = \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{(x_i - \mu)^2}{2\sigma^2}\right]$$

Step 2: Log-Likelihood Function

Taking the natural logarithm,

$$\ell(\mu) = -\frac{n}{2}\ln(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}(x_i - \mu)^2$$

Step 3: Differentiate with Respect to μ

$$\frac{d\ell(\mu)}{d\mu} = \frac{1}{\sigma^2}\sum_{i=1}^{n}(x_i - \mu)$$

Set the derivative equal to zero:

$$\sum_{i=1}^{n}(x_i - \mu) = 0$$

$$\sum_{i=1}^{n} x_i - n\mu = 0$$

Hence,

$$\boxed{\hat{\mu}_{MLE} = \frac{1}{n}\sum_{i=1}^{n} x_i}$$

Thus, **the MLE of the mean is simply the sample mean.**

Effect of Outliers

An **outlier** is an observation that is much larger or smaller than the rest of the data.

Since

$$\hat{\mu} = \frac{x_1 + x_2 + \cdots + x_n}{n},$$

every observation contributes equally to the estimate.

Suppose the data are:

$$\{10, , 11, , 9\ 10\ 100\}$$

Then,

$$\hat{\mu} = \frac{10 + 11 + 9 + 10 + 100}{5} = 28$$

Although four observations are close to 10, one extreme value shifts the estimate to **28**.

Analysis

- A single large outlier can significantly change the estimated mean.
- The MLE gives equal weight to every observation.
- Therefore, the estimator is highly sensitive to extreme values.

Effect of Increasing Sample Size

As the number of observations $n$increases,

$$\hat{\mu} = \frac{1}{n}\sum_{i=1}^{n} x_i$$

becomes a more accurate estimate of the true mean.

According to the **Law of Large Numbers**,

$$\hat{\mu} \to \mu \text{ as } n \to \infty$$

This means:

- Random fluctuations decrease.
- The variance of the estimator becomes smaller.

Specifically,

$$\boxed{\text{Var}(\hat{\mu}) = \frac{\sigma^2}{n}}$$

Hence,

- Larger sample size → Lower variance.
- Larger sample size → More precise estimate.
- Larger sample size → Better convergence to the true mean.

Implications for the Robustness of MLE

The robustness of an estimator refers to how well it performs when the data contain unusual or extreme observations.

Advantages

- **Efficient:** Produces minimum-variance estimates under Gaussian assumptions.

- **Consistent:** Converges to the true parameter as the sample size increases.
- **Unbiased:** The expected value of the estimator equals the true mean.

Limitations

- Highly sensitive to outliers.
- Even one extreme observation can noticeably alter the estimate.
- Performs best only when the data closely follow a normal distribution.

Conclusion

For a Gaussian distribution with known variance, the Maximum Likelihood Estimator of the mean is

$$\boxed{\hat{\mu}_{MLE} = \frac{1}{n}\sum_{i=1}^{n} x_i}$$

The estimate is simply the **sample mean**. When outliers are present, the estimate can shift significantly because all observations receive equal weight, making MLE **not robust to outliers**. As the sample size increases, however, the estimator becomes increasingly accurate and its variance decreases, making MLE **consistent and efficient** under the assumption of normally distributed data.

3. Building a Machine Learning Algorithm (Model Selection)

You are designing a classification algorithm for a dataset with 10,000 features but only 500 training samples.

Question:

Analyze the challenges involved in training such a model. Propose a strategy (algorithm pipeline) to handle this situation and justify each step (e.g., feature selection, regularization, dimensionality reduction). Explain how your approach mitigates overfitting.

ANSWER:

A dataset with **10,000 features** but only **500 training samples** is known as a **high-dimensional, low-sample-size (HDLSS)** problem. Here, the number of features is much larger than the number of training examples, making the model prone to overfitting.

Challenges

1. Overfitting

- With many features and few samples, the model can memorize the training data instead of learning general patterns.
- This results in high training accuracy but poor performance on unseen data.

2. Curse of Dimensionality

- As the number of features increases, the feature space becomes sparse.
- Distance-based algorithms become less effective, and much more data is required to learn meaningful patterns.

3. High Computational Cost

- Processing 10,000 features increases training time, memory usage, and computational complexity.

4. Irrelevant and Redundant Features

- Many features may contain little or no useful information.
- Redundant features introduce noise and reduce model performance.

5. Multicollinearity

- Highly correlated features make parameter estimation unstable and increase the variance of the model.

Proposed Machine Learning Pipeline

Step 1: Data Preprocessing

- Handle missing values.
- Normalize or standardize numerical features.
- Encode categorical variables if present.

Justification:

Standardized features improve optimization and ensure that all features contribute equally.

Step 2: Feature Selection

Use methods such as:

- Variance Threshold
- Correlation-based Feature Selection
- Mutual Information
- LASSO (L1 Regularization)
- Recursive Feature Elimination (RFE)

Reduce the feature set from **10,000 to a few hundred informative features**.

Justification:

- Removes irrelevant features.

- Reduces computational cost.
- Improves interpretability.
- Decreases overfitting.

Step 3: Dimensionality Reduction

Apply **Principal Component Analysis (PCA)** after feature selection.

PCA transforms correlated features into a smaller number of uncorrelated principal components while preserving most of the variance.

Justification:

- Eliminates redundancy.
- Reduces dimensionality further.
- Speeds up training.
- Helps the classifier focus on important information.

Step 4: Regularization

Use models with regularization such as:

- Logistic Regression with **L1 or L2 Regularization**
- Linear SVM with regularization

Regularization penalizes large model coefficients.

Justification:

- Prevents overly complex models.
- Reduces variance.
- Improves generalization.

Step 5: Model Selection

Suitable classifiers include:

- Logistic Regression (L1/L2)
- Linear Support Vector Machine (SVM)
- Naïve Bayes (for suitable data)

Avoid highly complex models such as deep neural networks because only **500 training samples** are available.

Justification:

Simple models generally generalize better on small datasets.

Step 6: Cross-Validation

Use **k-Fold Cross-Validation** (e.g., 5-fold or 10-fold).

Justification:

- Makes efficient use of limited data.
- Provides a reliable estimate of model performance.
- Helps in selecting the best hyperparameters.

Step 7: Hyperparameter Tuning

Use **Grid Search** or **Random Search** to optimize parameters such as:

- Regularization strength
- Number of selected features
- Number of principal components

Justification:

Improves model performance without increasing complexity unnecessarily.

How This Pipeline Mitigates Overfitting

Technique	How it Prevents Overfitting
Feature Selection	Removes irrelevant and noisy features, reducing model complexity.
Dimensionality Reduction (PCA)	Compresses data into fewer informative components and removes redundancy.
Regularization (L1/L2)	Penalizes large coefficients, preventing the model from fitting noise.
Simple Classifier	Lower complexity reduces the chance of memorizing training data.
Cross-Validation	Ensures the model generalizes well to unseen data and detects overfitting.
Hyperparameter Tuning	Selects the optimal model complexity for better generalization.

Conclusion

Training a classifier with **10,000 features and only 500 samples** is challenging due to the **curse of dimensionality**, **high computational cost**, and **severe risk of overfitting**. An effective pipeline consists of **data preprocessing**, **feature selection**, **dimensionality reduction (PCA)**, **regularized classification**, **cross-validation**, and **hyperparameter tuning**. This approach reduces noise, lowers model complexity, improves computational efficiency, and enables the model to generalize better to unseen data, thereby significantly mitigating overfitting.

A dataset with **10,000 features** but only **500 training samples** is known as a **high-dimensional, low-sample-size (HDLSS)** problem. Here, the number of features is much larger than the number of training examples, making the model prone to overfitting.

Challenges

1. Overfitting

- With many features and few samples, the model can memorize the training data instead of learning general patterns.
- This results in high training accuracy but poor performance on unseen data.

2. Curse of Dimensionality

- As the number of features increases, the feature space becomes sparse.
- Distance-based algorithms become less effective, and much more data is required to learn meaningful patterns.

3. High Computational Cost

- Processing 10,000 features increases training time, memory usage, and computational complexity.

4. Irrelevant and Redundant Features

- Many features may contain little or no useful information.
- Redundant features introduce noise and reduce model performance.

5. Multicollinearity

- Highly correlated features make parameter estimation unstable and increase the variance of the model.

Proposed Machine Learning Pipeline

Step 1: Data Preprocessing

- Handle missing values.
- Normalize or standardize numerical features.
- Encode categorical variables if present.

Justification:

Standardized features improve optimization and ensure that all features contribute equally.

Step 2: Feature Selection

Use methods such as:

- Variance Threshold
- Correlation-based Feature Selection
- Mutual Information
- LASSO (L1 Regularization)
- Recursive Feature Elimination (RFE)

Reduce the feature set from **10,000 to a few hundred informative features**.

Justification:

- Removes irrelevant features.
- Reduces computational cost.
- Improves interpretability.

- Decreases overfitting.

Step 3: Dimensionality Reduction

Apply **Principal Component Analysis (PCA)** after feature selection.

PCA transforms correlated features into a smaller number of uncorrelated principal components while preserving most of the variance.

Justification:

- Eliminates redundancy.
- Reduces dimensionality further.
- Speeds up training.
- Helps the classifier focus on important information.

Step 4: Regularization

Use models with regularization such as:

- Logistic Regression with **L1 or L2 Regularization**
- Linear SVM with regularization

Regularization penalizes large model coefficients.

Justification:

- Prevents overly complex models.
- Reduces variance.
- Improves generalization.

Step 5: Model Selection

Suitable classifiers include:

- Logistic Regression (L1/L2)
- Linear Support Vector Machine (SVM)
- Naïve Bayes (for suitable data)

Avoid highly complex models such as deep neural networks because only **500 training samples** are available.

Justification:

Simple models generally generalize better on small datasets.

Step 6: Cross-Validation

Use **k-Fold Cross-Validation** (e.g., 5-fold or 10-fold).

Justification:

- Makes efficient use of limited data.
- Provides a reliable estimate of model performance.

- Helps in selecting the best hyperparameters.

Step 7: Hyperparameter Tuning

Use **Grid Search** or **Random Search** to optimize parameters such as:

- Regularization strength
- Number of selected features
- Number of principal components

Justification:

Improves model performance without increasing complexity unnecessarily.

How This Pipeline Mitigates Overfitting

Technique	How it Prevents Overfitting
Feature Selection	Removes irrelevant and noisy features, reducing model complexity.
Dimensionality Reduction (PCA)	Compresses data into fewer informative components and removes redundancy.
Regularization (L1/L2)	Penalizes large coefficients, preventing the model from fitting noise.
Simple Classifier	Lower complexity reduces the chance of memorizing training data.
Cross-Validation	Ensures the model generalizes well to unseen data and detects overfitting.
Hyperparameter Tuning	Selects the optimal model complexity for better generalization.

Conclusion

Training a classifier with **10,000 features and only 500 samples** is challenging due to the **curse of dimensionality**, **high computational cost**, and **severe risk of overfitting**. An effective pipeline consists of **data preprocessing**, **feature selection**, **dimensionality reduction (PCA)**, **regularized classification**, **cross-validation**, and **hyperparameter tuning**. This approach reduces noise, lowers model complexity, improves computational efficiency, and enables the model to generalize better to unseen data, thereby significantly mitigating overfitting.

4. Neural Networks (Multilayer Perceptron - MLP)

Consider a Multilayer Perceptron with one hidden layer using sigmoid activation functions.

Question:

Explain analytically why adding more hidden neurons increases the representational power of the

network. Then, reason why this does not always lead to better performance. Support your answer using concepts like overfitting and generalization.

ANSWER :

A **Multilayer Perceptron (MLP)** is a type of artificial neural network consisting of an **input layer**, one or more **hidden layers**, and an **output layer**. In this network, the hidden layer uses the **sigmoid activation function**, which introduces non-linearity and enables the model to learn complex relationships.

Why More Hidden Neurons Increase Representational Power

Each hidden neuron learns a different feature or pattern from the input data. When more hidden neurons are added, the network can capture more complex relationships between the input and output.

- More neurons allow the network to learn a larger variety of features.
- The network can model complex and non-linear decision boundaries.
- This increases the ability of the MLP to approximate complicated functions.

Therefore, increasing the number of hidden neurons improves the **representational power** of the network.

Why More Hidden Neurons Do Not Always Improve Performance

Although more hidden neurons make the network more powerful, they do not always produce better results.

1. Overfitting

If the network has too many hidden neurons compared to the amount of training data, it may **memorize the training data** instead of learning general patterns. This results in:

- Very high training accuracy.
- Poor accuracy on new or unseen data.

2. Poor Generalization

A good model should perform well on both training and testing data. An excessively large network often learns the noise in the training data, reducing its ability to **generalize** to new data.

3. Increased Computational Cost

More hidden neurons increase:

- Training time.
- Memory usage.
- Number of parameters to optimize.

This makes the model slower and more computationally expensive.

Conclusion

Adding more hidden neurons increases the representational power of an MLP because it enables the network to learn more complex patterns and non-linear relationships. However, having too many hidden neurons can lead to **overfitting**, poor **generalization**, and higher computational cost. Therefore, the number of hidden neurons should be chosen carefully to achieve a balance between learning capability and model performance.

5. Backpropagation, SGD & Curse of Dimensionality

A deep neural network is trained using Stochastic Gradient Descent (SGD) on a very high-dimensional dataset.

Question:

Analyze how the **curse of dimensionality** impacts:

- Gradient updates in backpropagation
- Convergence speed of SGD
- Model generalization

Propose at least two techniques to overcome these issues and justify them analytically.

ANSWER

A deep neural network is trained using **Backpropagation** and **Stochastic Gradient Descent (SGD)** on a **very high-dimensional dataset**.

Backpropagation

Backpropagation is the learning algorithm used to train neural networks. It computes the error at the output layer and propagates it backward through the network to update the weights. This helps the network reduce the prediction error after each iteration.

Stochastic Gradient Descent (SGD)

SGD updates the network weights using **one training sample (or a small batch)** at a time instead of the entire dataset. This makes training faster and requires less memory. It also helps the model escape some local minima due to its random updates.

Curse of Dimensionality

A high-dimensional dataset contains a very large number of features. As the number of features increases:

- The data becomes sparse.
- More training samples are required to learn meaningful patterns.
- Training becomes slower and computationally expensive.

- The model is more likely to **overfit** by learning noise instead of useful patterns.

Challenges

- High computational cost.
- Slow convergence during training.
- Increased risk of overfitting.
- Poor generalization on unseen data.

How to Handle These Challenges

- **Feature Selection:** Remove irrelevant and redundant features.
- **Dimensionality Reduction (PCA):** Reduce the number of features while preserving important information.
- **Regularization (L1/L2):** Prevents the model from learning noise.
- **Dropout:** Randomly deactivates some neurons during training to reduce overfitting.
- **Mini-batch SGD:** Provides faster and more stable learning than using single samples.

Conclusion

Backpropagation and SGD efficiently train deep neural networks, but high-dimensional data introduces the **curse of dimensionality**, increasing computation and the risk of overfitting. Using **feature selection, dimensionality reduction, regularization, and dropout** improves learning, reduces overfitting, and helps the model generalize better to new data.